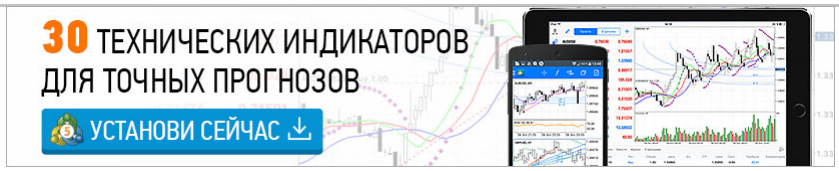
Tradays
★★★★½
Экономический календарь
Вперед



Нейросети — это просто (Часть 60): Онлайн Трансформер решений (Online Decision Transformer—ODT)

0 Dmitriy Gizlyk 1 695

Введение

Последние 2 статьи были посвящены методу Decision Transformer, который моделирует последовательности действий в контексте авторегрессионной модели желаемых вознаграждений. И мы остановились на том, что по результатам практических тестов 2 статей замечен довольно хороший рост доходности результатов работы обученной модели в начале периода тестирования. Далее результативность работы модели снижается и наблюдается ряд убыточных сделок, что ведет к потерям. Размер полученных убытков может превышать ранее полученную прибыль.

В подобной ситуации, вполне вероятно, может помочь периодическое дообучение модели. Однако, такой подход сильно усложняет процесс эксплуатации модели. И вполне резонно рассмотреть вариант онлайн обучения модели. Но здесь мы сталкиваемся с рядом задач, которые нам предстоит решить.

Один из вариантов реализации онлайн обучения Decision Transformer представлен в статье "Online Decision Transformer" (февраль 2022 года). Стоит сразу отметить, что предложенный метод использует первичное офлайн обучение классического DT. Онлайн обучение применяется в процессе последующей тонкой настройки модели. Представленные в авторской статье результаты экспериментов демонстрируют, что ODT способен конкурировать с лидерами в абсолютной производительности на тестовой выборке D4RL. При этом он показывает гораздо более значительное улучшение в процессе настройки.

Давайте рассмотрим предложенный метод в контексте решения наших задач.

1. Алгоритм ODT

Перед рассмотрением алгоритма Online Decision Transformer предлагаю вкратце вспомнить классический Трансформер решений. DT обрабатывает траекторию τ как последовательность из нескольких входных токенов: Return-to-Go (RTG), состояний и действий. В частности, начальное значение RTG равно возврату для всей траектории. На временном шаге t DT использует токены K последних временных шагов, чтобы сгенерировать действие A_t. В данном случае K является гиперпараметром, который определяет длину контекста для трансформера. Важно отметить, что во время эксплуатации длина контекста может быть короче, чем во время обучения.

DT обучается детерминированной политике $\pi(A_t|S_t, RTG_t)$, где S_t представляет собой последовательность K последних состояний от $t-K+1$ до t. Аналогично $RTGt$ олицетворяет K последних Return-to-Go. Это авторегрессионная модель K-порядка. Политика Агента обучается прогнозировать действия с использованием стандартной функции потерь MSE (средне-квадратическая ошибка).

В процессе эксплуатации мы указываем желаемую производительность $RTG_{нач}$ и начальное состояние S_0. DT затем генерирует действие A_0. После генерации действия A_t мы выполняем его и наблюдаем

следующее состояние S_{t+1}, получая вознаграждение r_t. Это дает нам следующее RTG_{t+1}.

$$RTG_{t+1} = RTG_t - r_t$$

Как и ранее, DT генерирует действие A_1 на основании траектории, включающей A_0, S_0, S_1 и RTG_0, RTG_1. Этот процесс повторяется до завершения эпизода.

Политики, обученные только на офлайн наборах данных, обычно являются неоптимальными из-за ограниченных данных обучающей выборки. Офлайн траектории могут иметь низкий возврат и охватывать только ограниченную часть пространства состояний и действий. Естественной стратегией для улучшения производительности является дообучение Агентов RL в онлайн взаимодействии с окружающей средой. Но стандартный метод Decision Transformer недостаточен для онлайн обучения.

Алгоритм Online Decision Transformer вводит ключевые модификации Decision Transformer для обеспечения эффективного онлайн обучения. И первым шагом является обобщенная вероятностная цель обучения. В данном контексте цель — обучить стохастическую политику, которая максимизирует вероятность повторения траектории.

Основное свойство онлайн RL-алгоритма — это способность сбалансировать компромисс между исследованием и эксплуатацией. Даже со стохастическими политиками, традиционная формулировка DT не учитывает исследование. Для решения этой проблемы авторами метода ODT исследование определяется через энтропию политики, которая зависит от распределения данных в траектории. Это распределение статично во время предварительного офлайн обучения, но динамично во время онлайн настройки, так как оно зависит от новых данных, полученных во время взаимодействия с окружающей средой.

Аналогично многим существующим RL-алгоритмам с максимальной энтропией, таким как Soft Actor Critic, авторы метода ODT явно определяют нижний предел энтропии политики, чтобы поощрить исследование.

Отличие функции потерь ODT от SAC и других классических методов RL заключается в том, что в ODT функция потерь — это отрицательное логарифмическое правдоподобие, а не дисконтированный возврат. В принципе, мы фокусируемся только на обучении по образцу последовательности действий, вместо явной максимизации возврата. И целевая функция автоматически адаптируется для подходящей политики Актера как в офлайн, так и в онлайн обучении. Во время офлайн обучения кросс-энтропия контролирует степень несовпадения распределения, а во время онлайн обучения она стимулирует политику исследования.

Еще одно важное отличие от классических методов максимальной энтропии RL заключается в том, что в ODT энтропия политики определена на уровне последовательностей, а не переходов. В то время как SAC накладывает нижний предел β для энтропии политики на всех временных шагах, ODT ограничивает энтропию усреднено на K последовательных временных шагах. Таким образом, ограничение требует только, чтобы энтропия, усредненная по последовательности из K временных шагов, была выше установленного значения β. Поэтому любая политика, удовлетворяющая ограничению на уровне переходов, также удовлетворяет ограничению на уровне последовательности. Таким образом, пространство допустимых политик больше, когда $K > 1$. Когда $K = 1$, ограничение на уровне последовательности сокращается до ограничения на уровне переходов аналогично SAC.

В процессе обучения моделей используется буфер воспроизведения для записи предыдущего опыта с периодическим обновлением. Для большинства существующих алгоритмов RL буфер воспроизведения опыта состоит из переходов. После каждого этапа онлайн взаимодействия в рамках одной эпохи, политика Агента и Q-функция обновляются с помощью градиентного спуска. Затем политика выполняется для сбора новых переходов и добавления их в буфер воспроизведения опыта. В случае ODT, буфер воспроизведения опыта состоит из траекторий, а не переходов. После предварительного офлайн обучения мы инициализируем буфер воспроизведения опыта траекториями с максимальными результатами из офлайн набора данных. Каждый раз при взаимодействии с окружающей средой мы полностью выполняем эпизод с текущей политикой. После чего обновляем буфер воспроизведения опыта с использованием собранной траектории в порядке FIFO. А затем обновляем политику Агента и

выполняем новый эпизод. Оценка политики с использованием среднего действия обычно приводит к более высоким вознаграждениям, но полезно использовать случайные действия для онлайн исследования, так как это генерирует более разнообразные траектории и поведенческие модели.

Кроме того, алгоритму ODT необходим гиперпараметр в виде начального *RTG* для сбора дополнительных онлайн данных. Различные работы демонстрируют, что фактический оценочный возврат офлайн DT имеет сильную корреляцию с начальным *RTG* эмпирически и часто может экстраполировать значения *RTG* за пределами максимальных возвратов, наблюдаемых в офлайн наборе данных. Авторы ODT обнаружили, что лучше всего установить этот гиперпараметр с небольшим фиксированным масштабирование от имеющихся результатов эксперта. Авторы метода в своей работе используют 2 кратное масштабирование. В оригинальной статье приведены результаты экспериментов с гораздо большими значениями, а также с изменяющимися во время обучения (например, квантилями лучшего оценочного возврата в офлайн и онлайн наборах данных). Но на практике они оказались не так эффективны, как фиксированный масштабированный RTG.

Как и DT, Алгоритм ODT использует двух-шаговую процедуру выборки, чтобы обеспечить равномерную выборку под-траекторий длиной *K* в буфере воспроизведения. Сначала семплируем одну траекторию с вероятностью, пропорциональной ее длине. Затем равновероятно выбираем под-траекторию длиной *K*.

С практической реализацией метода мы познакомимся в следующем разделе данной статьи.

2. Реализация средствами MQL5

После ознакомления с теоретическими аспектами метода мы переходим к его практической реализации. В данном разделе будет представлено собственное видение реализации предложенных подходов, дополненных наработками из предыдущих статей. В частности, алгоритм ODT включает двух-этапное обучение модели:

1. Предварительное офлайн обучение.
2. Тонкая настройка модели во время онлайн взаимодействия с окружающей средой.

В рамках данной статьи мы будем использовать предварительно обученную модель из предыдущей статьи. Следовательно, пропускаем первый этап офлайн обучения, который уже проведен ранее. И сразу переходим ко второй части процесса обучения модели.

Здесь же следует обратить, что при рассмотрении метода DoC в предыдущей статье мы построили и провели офлайн обучение 2 моделей:

- генерации *RTG*;
- политики Актера.

Использование модели генерации *RTG* является отступлением от авторского алгоритма ODT, в котором предлагается использование масштабирование экспертной оценки для начального RTG с последующей корректировкой цели на фактически полученные результаты.

Кроме того, использование ранее обученных моделей не позволяет нам изменить архитектуру моделей. Но давайте посмотрим, насколько архитектура используемых моделей соответствует алгоритму ODT.

Авторы метода предлагают использовать стохастическую политику Актера. Именно такую мы использовали в предыдущих статьях. В этом плане мы соответствуем.

В ODT предлагается использовать буфер воспроизведения опыта траекторий вместо отдельных траекторий. Именно с таким буфером мы работаем. Здесь мы тоже попадаем.

Должен сказать, что при обучении моделей мы не использовали энтропийную составляющую функции потерь для стимулирования исследования окружающей среды. На данном этапе мы не будем её добавлять и принимаем возможные риски. Мы рассчитываем, что стохастическая политика Актера и модель генерации RTG обеспечат достаточное исследование в процессе онлайн взаимодействия с окружающей средой.

Ещё один момент, который я исключил из своей реализации, касается буфера воспроизведения опыта. Авторы метода предлагают после офлайн обучения отобрать некоторое количество наиболее доходных траекторий, которые будут использоваться на первых этапах онлайн обучения. Мы же изначально ограничили количество траекторий в буфере воспроизведения опыта. И при переходе к онлайн обучению мы будем использовать весь имеющийся буфер воспроизведения опыта, в который будем добавлять новые траектории в процессе взаимодействия с окружающей средой. При этом мы не будем сразу удалять наиболее старые траектории при добавлении новых. Ограничивать размер буфера мы будем ранее созданными средствами при сохранении данных в файл после завершения прохода.

Таким образом, с учетом принятия возможных рисков мы вполне можем использовать модели, обученные в рамках предыдущей статьи. И далее мы попробуем повысить их эффективность путем тонкой настройки в процессе онлайн обучения моделей с использованием подходов ODT.

И здесь нам предстоит решить некоторые конструктивные вопросы. Процесс трейдинга по своей природе условно бесконечен. Условно, потому что по ряду причин он все же конечен. Но вероятность наступления такого события настолько мала в обозримом будущем, что мы считаем его бесконечным. Следовательно, процесс дообучения мы осуществляем не по завершении эпизода, как предлагается авторами метода, а с определенной периодичностью.

И тут хочу напомнить, что в нашей реализации DT на вход модели подаются только данные последнего бара. Весь объем контекста исторических данных хранится в буфере результатов слоя эмбединга. Такой подход позволил нам сократить потребление ресурсов на избыточную повторную обработку данных. Но это становится одним из "камней преткновения" на пути онлайн обучения. Дело в том, что данные в буфере эмбединга хранятся в строгой исторической последовательности. А использование модели в процессе периодического дообучения ведет к перезаполнению буфера историческими данными из других траекторий или этой же траектории, но другого отрезка истории. Что исказит данные при продолжении взаимодействия со средой после дообучения моделей.

Вариантов решений данного вопроса на самом деле несколько. Все имеют различную сложность реализации и потребление ресурсов в процессе эксплуатации. На первый взгляд, наиболее простым является создание копии буфера и перед продолжением процесса взаимодействия со средой вернуть буферу состояние до запуска процесса обучения. Однако, при детальном рассмотрении процесса мы понимаем, что на стороне основной модели осуществляется работа только с верхнеуровневым классом модели без доступа к отдельным буферам нейронных слоев. И в этом контексте простой процесс копирования данных одного буфера из модели и в модель обратно ведет к целому ряду конструктивных изменений. Что значительно усложняет реализацию данного метода.

Практически без внесения конструктивных изменений в модель мы можем после завершения процесса дообучения повторно последовательно передать в модель весь набор исторических данных. Но это ведет к значительному объему повторения операций прямого прохода модели. И объем таких операций растет по мере увеличения размера контекста. Что делает такой подход не эффективным. Расход времени и вычислительных ресурсов на повторную обработку данных может превысить достигнутую экономию при сохранении истории эмбедингов в буфере нейронного слоя.

Ещё один вариант решения проблемы лежит использовании дубликатов моделей. Одна для взаимодействия с окружающей средой. Вторая используется в процессе дообучения. Такой подход более затратный по ресурсам памяти, но полностью решает вопрос с данными в буфере слоя эмбединга. Но возникает вопрос обмена данными между моделями. Ведь после дообучения модель взаимодействия с окружающей средой должна использовать обновленную политику Агента. Аналогично и модель генерации RTG. И здесь можно вспомнить метод Soft Actor-Critic с его мягким обновлением целевых моделей. Как это не покажется странным, но именно такой механизм нам позволит передать обновленные весовые коэффициенты между моделями без изменения остальных буферов. В том числе и буфера результатов слоя эмбединга.

Для использования данного подхода нам предстоит добавить метод обмена весовыми коэффициентами в слой эмбединга, который ранее не использовался при реализации SAC.

Здесь надо сказать, что при добавлении метода мы вносим дополнения только непосредственно в класс CNeuronEmbeddingOCL, так как все необходимые API его функционирования уже заложены нами ранее и реализованы в виде виртуального метода базового класса нейронного слоя CNeuronBaseOCL. Надо так же обратить внимание, что без внесения указанной доработки эксплуатация нашей модели не выдаст ошибки. Ведь по умолчанию будет использоваться метод родительского класса. Но такая работа в данном случае будет не полной и не корректной.

Для сохранения наследовательности и корректного переопределения виртуальных методов мы объявляем метод с сохранением параметров. В теле метода мы сразу вызываем аналогичный метод родительского класса.

```
bool CNeuronEmbeddingOCL::WeightsUpdate(CNeuronBaseOCL *source, float tau)
  {
   if(!CNeuronBaseOCL::WeightsUpdate(source, tau))
      return false;
```

Как мы уже не раз говорили, подобный подход вызова родительского класса позволяет нам одним действием осуществить все необходимы контроли без излишнего их дублирования и выполнить необходимые операции с унаследованными объектами.

После успешного выполнения операций метода родительского класса мы переходим к работе над объектами, объявленными непосредственно в нашем классе эмбединга. Но чтобы получить доступ к аналогичным объектам донорского класса мы должны переопределить тип полученного объекта.

```
//---
   CNeuronEmbeddingOCL *temp = source;
```

Далее нам предстоит перенести параметры буфера WeightsEmbedding. Но перед продолжением операций мы сравним размеры буферов текущего и донорского объекта.

```
   if(WeightsEmbedding.Total() != temp.WeightsEmbedding.Total())
      return false;
```

А затем нам предстоит перенести содержание из одного буфера в другой. Но мы помним, что все операции с буферами осуществляются на стороне контекста OpenCL. Поэтому и перенос данных мы будем осуществлять на стороне контекста. Я намеренно использую фразу "перенос данных", а не "копирование". Так как оставляю возможность "мягкого копирования" с коэффициентом, как это было предусмотрено алгоритмом SAC для целевых моделей. Кернелы OpenCL программы были созданы ранее. И сейчас нам предстоит лишь организовать их вызов.

Пространство задач кернела мы определяем в размере буфера весовых коэффициентов.

```
   uint global_work_offset[1] = {0};
   uint global_work_size[1] = {WeightsEmbedding.Total()};
```

Далее следует разветвление алгоритма в зависимости от используемого алгоритма обновления параметров. Разветвление необходимо, так как в случае использования метода Adam нам потребуется больше буферов и гиперпараметров. Что ведет к использованию различных кернелов.

Первой мы создаем ветку метода Adam. Для её использования необходимо соблюдение 2 условий:

- указание соответствующего метода обновления параметров при создании объекта, так как от этого зависит создание объектов соответствующих буферов данных;
- коэффициент обновления должен быть отличен от 1, в противном случае необходимо полное копирование данных вне зависимости от используемого метода обновления параметров.

В теле условного оператора мы заполняем параметры вызываемого кернела указателями на соответствующие буферы данных и необходимыми гиперпараметрами.

```
   if(tau != 1.0f && optimization == ADAM)
     {
      if(!OpenCL.SetArgumentBuffer(def_k_SoftUpdateAdam, def_k_sua_target, WeightsEmbed
        {
         printf("Error of set parameter kernel %s: %d; line %d", __FUNCTION__, GetLastE
         return false;
        }
      if(!OpenCL.SetArgumentBuffer(def_k_SoftUpdateAdam, def_k_sua_source, temp.Weights
        {
         printf("Error of set parameter kernel %s: %d; line %d", __FUNCTION__, GetLastE
         return false;
        }
      if(!OpenCL.SetArgumentBuffer(def_k_SoftUpdateAdam, def_k_sua_matrix_m, FirstMomer
        {
         printf("Error of set parameter kernel %s: %d; line %d", __FUNCTION__, GetLastE
         return false;
        }
      if(!OpenCL.SetArgumentBuffer(def_k_SoftUpdateAdam, def_k_sua_matrix_v, SecondMome
        {
         printf("Error of set parameter kernel %s: %d; line %d", __FUNCTION__, GetLastE
         return false;
        }
      if(!OpenCL.SetArgument(def_k_SoftUpdateAdam, def_k_sua_tau, (float)tau))
        {
         printf("Error of set parameter kernel %s: %d; line %d", __FUNCTION__, GetLastE
         return false;
        }
      if(!OpenCL.SetArgument(def_k_SoftUpdateAdam, def_k_sua_b1, (float)b1))
        {
         printf("Error of set parameter kernel %s: %d; line %d", __FUNCTION__, GetLastE
         return false;
        }
      if(!OpenCL.SetArgument(def_k_SoftUpdateAdam, def_k_sua_b2, (float)b2))
        {
         printf("Error of set parameter kernel %s: %d; line %d", __FUNCTION__, GetLastE
         return false;
        }
      if(!OpenCL.Execute(def_k_SoftUpdateAdam, 1, global_work_offset, global_work_size)
        {
         printf("Error of execution kernel %s: %d", __FUNCTION__, GetLastError());
         return false;
        }
     }
```

После чего отправляем кернел SoftUpdateAdam в очередь выполнения.

Аналогичный операции выполняем и во второй ветке алгоритма, но уже для кернела SoftUpdate.

```
   else
     {
      if(!OpenCL.SetArgumentBuffer(def_k_SoftUpdate, def_k_su_target, WeightsEmbedding.
        {
         printf("Error of set parameter kernel %s: %d; line %d", __FUNCTION__, GetLastE
         return false;
```

```
         }
       if(!OpenCL.SetArgumentBuffer(def_k_SoftUpdate, def_k_su_source, temp.WeightsEmbed
         {
          printf("Error of set parameter kernel %s: %d; line %d", __FUNCTION__, GetLastE
          return false;
         }
       if(!OpenCL.SetArgument(def_k_SoftUpdate, def_k_su_tau, (float)tau))
         {
          printf("Error of set parameter kernel %s: %d; line %d", __FUNCTION__, GetLastE
          return false;
         }
       if(!OpenCL.Execute(def_k_SoftUpdate, 1, global_work_offset, global_work_size))
         {
          printf("Error of execution kernel %s: %d", __FUNCTION__, GetLastError());
          return false;
         }
      }
//---
   return true;
  }
```

Конструктивная задача решена, и мы можем переходить к практической реализации метода онлайн обучения. Процесс взаимодействия с окружающей средой и одновременного дообучения моделей организуем в советнике «...\DoC\OnlineStudy.mq5». Данный советник является неким симбиозом рассмотренных в предыдущих статьях советников сбора данных для обучения и непосредственного офлайн обучения моделей. Он содержит все внешние параметры, необходимы для взаимодействия с окружающей средой. В частности, параметры индикаторов. Но при этом мы добавляем параметры для указания периодичности и количества итераций онлайн обучения. В советнике по умолчанию указаны субъективные данные. Периодичность обучения я указал в 120 свечей, что на таймфрейме H1 примерно соответствует 1 недели (5 дней * 24 часа). В процессе оптимизации Вы можете подобрать значения, которые будут более оптимальными для Ваших моделей.

```
//+------------------------------------------------------------------+
//| Input parameters                                                 |
//+------------------------------------------------------------------+
input ENUM_TIMEFRAMES      TimeFrame    =  PERIOD_H1;
//---
input group                "---- RSI ----"
input int                  RSIPeriod    =  14;            //Period
input ENUM_APPLIED_PRICE   RSIPrice     =  PRICE_CLOSE;   //Applied price
//---
input group                "---- CCI ----"
input int                  CCIPeriod    =  14;            //Period
input ENUM_APPLIED_PRICE   CCIPrice     =  PRICE_TYPICAL; //Applied price
//---
input group                "---- ATR ----"
input int                  ATRPeriod    =  14;            //Period
//---
input group                "---- MACD ----"
input int                  FastPeriod   =  12;            //Fast
input int                  SlowPeriod   =  26;            //Slow
input int                  SignalPeriod=    9;            //Signal
input ENUM_APPLIED_PRICE   MACDPrice    =  PRICE_CLOSE;   //Applied price
//---
input int                  StudyIters   =   5;            //Iterations to Study
input int                  StudyPeriod  =  120;           //Bars between Studies
```

В методе инициализации советника мы сначала загружаем ранее созданный буфер воспроизведения опыта. Подобные действия мы выполняли в советниках обучения "Study.mql5" различных методов офлайн обучения. Только сейчас мы не завершаем работу советника при неудачной загрузке данных. В отличии от офлайн режима, мы допускаем обучение моделей только на новых данных, которые будут собраны при взаимодействии с окружающей средой.

```
//+------------------------------------------------------------------+
//| Expert initialization function                                   |
//+------------------------------------------------------------------+
int OnInit()
  {
   LoadTotalBase();
```

Далее мы подготовим индикаторы, так же как это делали ранее в советниках взаимодействия с окружающей средой.

```
   if(!Symb.Name(_Symbol))
      return INIT_FAILED;
   Symb.Refresh();
//---
   if(!RSI.Create(Symb.Name(), TimeFrame, RSIPeriod, RSIPrice))
      return INIT_FAILED;
//---
   if(!CCI.Create(Symb.Name(), TimeFrame, CCIPeriod, CCIPrice))
      return INIT_FAILED;
//---
   if(!ATR.Create(Symb.Name(), TimeFrame, ATRPeriod))
      return INIT_FAILED;
//---
   if(!MACD.Create(Symb.Name(), TimeFrame, FastPeriod, SlowPeriod, SignalPeriod, MACDPr
      return INIT_FAILED;
   if(!RSI.BufferResize(NBarInPattern) || !CCI.BufferResize(NBarInPattern) ||
      !ATR.BufferResize(NBarInPattern) || !MACD.BufferResize(NBarInPattern))
     {
      PrintFormat("%s -> %d", __FUNCTION__, __LINE__);
      return INIT_FAILED;
     }
//---
   if(!Trade.SetTypeFillingBySymbol(Symb.Name()))
      return INIT_FAILED;
```

Загрузим модели и проверим их соответствие в части размеров слоев исходных данных и результатов. При необходимости мы создаем новые модели с заранее определенной архитектурой. Это немного выходим за рамки дообучения моделей. Но мы оставляем возможность пользователю осуществлять онлайн обучение "с нуля".

```
//--- load models
   float temp;
   if(!Agent.Load(FileName + "Act.nnw", temp, temp, temp, dtStudied, true) ||
      !RTG.Load(FileName + "RTG.nnw", dtStudied, true) ||
      !AgentStudy.Load(FileName + "Act.nnw", temp, temp, temp, dtStudied, true) ||
      !RTGStudy.Load(FileName + "RTG.nnw", dtStudied, true))
     {
      PrintFormat("Can't load pretrained models");
      CArrayObj *agent = new CArrayObj();
      CArrayObj *rtg = new CArrayObj();
```

```
      if(!CreateDescriptions(agent, rtg))
        {
         delete agent;
         delete rtg;
         PrintFormat("Can't create description of models");
         return INIT_FAILED;
        }
      if(!Agent.Create(agent) ||
         !RTG.Create(rtg) ||
         !AgentStudy.Create(agent) ||
         !RTGStudy.Create(rtg))
        {
         delete agent;
         delete rtg;
         PrintFormat("Can't create models");
         return INIT_FAILED;
        }
      delete agent;
      delete rtg;
      //---
     }
//---
   Agent.getResults(Result);
   if(Result.Total() != NActions)
     {
      PrintFormat("The scope of the actor does not match the actions count (%d <> %d)",
      return INIT_FAILED;
     }
   AgentResult = vector<float>::Zeros(NActions);
//---
   Agent.GetLayerOutput(0, Result);
   if(Result.Total() != (NRewards + BarDescr * NBarInPattern + AccountDescr + TimeDescr
     {
      PrintFormat("Input size of Actor doesn't match state description (%d <> %d)",
           Result.Total(), (NRewards + BarDescr * NBarInPattern + AccountDescr + TimeDe
      return INIT_FAILED;
     }
   Agent.Clear();
   RTG.Clear();
```

Обратите внимание, что мы загружаем (или инициализируем) по 2 копии каждой модели. Одна для взаимодействия с окружающей средой. А вторая используется в процессе обучения. Обучаемые модели получили суффикс Study.

Затем инициализируем глобальные переменные и завершаем работу метода.

```
   PrevBalance = AccountInfoDouble(ACCOUNT_BALANCE);
   PrevEquity = AccountInfoDouble(ACCOUNT_EQUITY);
//---
   return(INIT_SUCCEEDED);
  }
```

В методе деинициализации советника мы сохраняем обучаемые модели и накопленный буфер воспроизведения опыта.

```
//+------------------------------------------------------------------+
//| Expert deinitialization function                                 |
```

```
//+------------------------------------------------------------------+
void OnDeinit(const int reason)
  {
//---
   AgentStudy.Save(FileName + "Act.nnw", 0, 0, 0, TimeCurrent(), true);
   RTGStudy.Save(FileName + "RTG.nnw", TimeCurrent(), true);
   delete Result;
   int total = ArraySize(Buffer);
   printf("Saving %d", MathMin(total + 1, MaxReplayBuffer));
   SaveTotalBase();
   Print("Saved");
  }
```

Обратите внимание, что сохраняем именно обучаемые модели. Так как их буферы содержат всю информацию, необходимую для последующего обучения и эксплуатации моделей.

Процесс взаимодействия с окружающей средой организован в методе обработки тиков OnTick. В начале метода мы проверяем наступления события открытия нового бара и, при необходимости, обновляем параметры индикаторов. А также загружаем данные ценового движения.

```
//+------------------------------------------------------------------+
//| Expert tick function                                             |
//+------------------------------------------------------------------+
void OnTick()
  {
//---
   if(!IsNewBar())
      return;
//---
   int bars = CopyRates(Symb.Name(), TimeFrame, iTime(Symb.Name(), TimeFrame, 1), NBarI
   if(!ArraySetAsSeries(Rates, true))
      return;
//---
   RSI.Refresh();
   CCI.Refresh();
   ATR.Refresh();
   MACD.Refresh();
   Symb.Refresh();
   Symb.RefreshRates();
```

Полученные от терминала данные мы подготавливаем для передачи в модель взаимодействия с окружающей средой в качестве исходных данных.

```
//--- History data
   float atr = 0;
   for(int b = 0; b < (int)NBarInPattern; b++)
     {
      float open = (float)Rates[b].open;
      float rsi = (float)RSI.Main(b);
      float cci = (float)CCI.Main(b);
      atr = (float)ATR.Main(b);
      float macd = (float)MACD.Main(b);
      float sign = (float)MACD.Signal(b);
      if(rsi == EMPTY_VALUE || cci == EMPTY_VALUE || atr == EMPTY_VALUE || macd == EMPT
         continue;
      //---
      int shift = b * BarDescr;
```

```
      sState.state[shift] = (float)(Rates[b].close - open);
      sState.state[shift + 1] = (float)(Rates[b].high - open);
      sState.state[shift + 2] = (float)(Rates[b].low - open);
      sState.state[shift + 3] = (float)(Rates[b].tick_volume / 1000.0f);
      sState.state[shift + 4] = rsi;
      sState.state[shift + 5] = cci;
      sState.state[shift + 6] = atr;
      sState.state[shift + 7] = macd;
      sState.state[shift + 8] = sign;
     }
   bState.AssignArray(sState.state);
```

Дополним буфер данных информацией о состоянии счета.

```
//--- Account description
   sState.account[0] = (float)AccountInfoDouble(ACCOUNT_BALANCE);
   sState.account[1] = (float)AccountInfoDouble(ACCOUNT_EQUITY);
//---
   double buy_value = 0, sell_value = 0, buy_profit = 0, sell_profit = 0;
   double position_discount = 0;
   double multiplyer = 1.0 / (60.0 * 60.0 * 10.0);
   int total = PositionsTotal();
   datetime current = TimeCurrent();
   for(int i = 0; i < total; i++)
     {
      if(PositionGetSymbol(i) != Symb.Name())
         continue;
      double profit = PositionGetDouble(POSITION_PROFIT);
      switch((int)PositionGetInteger(POSITION_TYPE))
        {
         case POSITION_TYPE_BUY:
            buy_value += PositionGetDouble(POSITION_VOLUME);
            buy_profit += profit;
            break;
         case POSITION_TYPE_SELL:
            sell_value += PositionGetDouble(POSITION_VOLUME);
            sell_profit += profit;
            break;
        }
      position_discount += profit - (current - PositionGetInteger(POSITION_TIME)) * mul
     }
   sState.account[2] = (float)buy_value;
   sState.account[3] = (float)sell_value;
   sState.account[4] = (float)buy_profit;
   sState.account[5] = (float)sell_profit;
   sState.account[6] = (float)position_discount;
   sState.account[7] = (float)Rates[0].time;
//---
   bState.Add((float)((sState.account[0] - PrevBalance) / PrevBalance));
   bState.Add((float)(sState.account[1] / PrevBalance));
   bState.Add((float)((sState.account[1] - PrevEquity) / PrevEquity));
   bState.Add(sState.account[2]);
   bState.Add(sState.account[3]);
   bState.Add((float)(sState.account[4] / PrevBalance));
   bState.Add((float)(sState.account[5] / PrevBalance));
   bState.Add((float)(sState.account[6] / PrevBalance));
```

Далее мы создаем временную метку.

```
//--- Time label
   double x = (double)Rates[0].time / (double)(D'2024.01.01' - D'2023.01.01');
   bState.Add((float)MathSin(2.0 * M_PI * x));
   x = (double)Rates[0].time / (double)PeriodSeconds(PERIOD_MN1);
   bState.Add((float)MathCos(2.0 * M_PI * x));
   x = (double)Rates[0].time / (double)PeriodSeconds(PERIOD_W1);
   bState.Add((float)MathSin(2.0 * M_PI * x));
   x = (double)Rates[0].time / (double)PeriodSeconds(PERIOD_D1);
   bState.Add((float)MathSin(2.0 * M_PI * x));
```

И добавим вектор последних действий Агента, которые привели нас в текущее состояние.

```
//--- Prev action
   bState.AddArray(AgentResult);
```

Собранных данных достаточно для осуществления прямого прохода модели генерации RTG.

```
//--- Return to go
   if(!RTG.feedForward(GetPointer(bState)))
      return;
```

По сути, в нашем векторе исходных данных не хватает только этих данных для прогнозирования оптимальных действий Агента на текущем временном отрезке. Поэтому, после успешного прямого прохода первой модели, мы добавляем полученные результаты в буфер исходных данных и вызываем метод прямого прохода нашего Актера. Обязательно проверяем результат выполнения операций.

```
   RTG.getResults(Result);
   bState.AddArray(Result);
//---
   if(!Agent.feedForward(GetPointer(bState), 1, false, (CBufferFloat*)NULL))
      return;
```

После успешного выполнения прямого прохода моделей мы дешифруем результаты их работы и совершим выбранное действие в окружающей среде. Данный процесс полностью соответствует ранее рассмотренному алгоритму в моделях взаимодействия с окружающей средой.

```
//---
   PrevBalance = sState.account[0];
   PrevEquity = sState.account[1];
//---
   vector<float> temp;
   Agent.getResults(temp);
//---
   double min_lot = Symb.LotsMin();
   double step_lot = Symb.LotsStep();
   double stops = MathMax(Symb.StopsLevel(), 1) * Symb.Point();
   if(temp[0] >= temp[3])
     {
      temp[0] -= temp[3];
      temp[3] = 0;
     }
   else
     {
      temp[3] -= temp[0];
```

```
      temp[0] = 0;
    }
  float delta = MathAbs(AgentResult - temp).Sum();
  AgentResult = temp;
//--- buy control
  if(temp[0] < min_lot || (temp[1] * MaxTP * Symb.Point()) <= stops || (temp[2] * MaxS
    {
     if(buy_value > 0)
        CloseByDirection(POSITION_TYPE_BUY);
    }
  else
    {
     double buy_lot = min_lot + MathRound((double)(temp[0] - min_lot) / step_lot) * st
     double buy_tp = Symb.NormalizePrice(Symb.Ask() + temp[1] * MaxTP * Symb.Point());
     double buy_sl = Symb.NormalizePrice(Symb.Ask() - temp[2] * MaxSL * Symb.Point());
     if(buy_value > 0)
        TrailPosition(POSITION_TYPE_BUY, buy_sl, buy_tp);
     if(buy_value != buy_lot)
       {
        if(buy_value > buy_lot)
           ClosePartial(POSITION_TYPE_BUY, buy_value - buy_lot);
        else
           Trade.Buy(buy_lot - buy_value, Symb.Name(), Symb.Ask(), buy_sl, buy_tp);
       }
    }
//--- sell control
  if(temp[3] < min_lot || (temp[4] * MaxTP * Symb.Point()) <= stops || (temp[5] * MaxS
    {
     if(sell_value > 0)
        CloseByDirection(POSITION_TYPE_SELL);
    }
  else
    {
     double sell_lot = min_lot + MathRound((double)(temp[3] - min_lot) / step_lot) * s
     double sell_tp = Symb.NormalizePrice(Symb.Bid() - temp[4] * MaxTP * Symb.Point())
     double sell_sl = Symb.NormalizePrice(Symb.Bid() + temp[5] * MaxSL * Symb.Point())
     if(sell_value > 0)
        TrailPosition(POSITION_TYPE_SELL, sell_sl, sell_tp);
     if(sell_value != sell_lot)
       {
        if(sell_value > sell_lot)
           ClosePartial(POSITION_TYPE_SELL, sell_value - sell_lot);
        else
           Trade.Sell(sell_lot - sell_value, Symb.Name(), Symb.Bid(), sell_sl, sell_tp
       }
    }
```

Оцениваем вознаграждение от окружающей среды за переход в текущее состояние. И всю собранную информацию передаем для формирования текущей траектории.

```
//---
  int shift = BarDescr * (NBarInPattern - 1);
  sState.rewards[0] = bState[shift];
  sState.rewards[1] = bState[shift + 1] - 1.0f;
  if((buy_value + sell_value) == 0)
     sState.rewards[2] -= (float)(atr / PrevBalance);
  else
```

```
      sState.rewards[2] = 0;
   for(ulong i = 0; i < NActions; i++)
      sState.action[i] = AgentResult[i];
   if(!Base.Add(sState))
      ExpertRemove();
```

На этом завершается процесс взаимодействия с окружающей средой. Но перед выходом из метода мы проверим необходимость запуска процесса дообучения моделей. Для целей проверки эффективности метода я использовал, наверное, самый простой контроль. Я просто проверяю кратность размера общей истории по инструменту в анализируемом таймфрейме периодичности обучения. В условиях промышленной эксплуатации желательно использовать более продуманные подходы, чтобы сместить процесс дообучения на периоды закрытия рынка или снижения волатильности по инструменту. Кроме того, возможно будет полезно отложить процесс обновления параметров моделей до закрытия всех позиций. В целом, для использования в реальных моделях я бы рекомендовал более взвешенно и осмысленно подходить к выбору периодичности и времени дообучения моделей.

```
//---
   if((Bars(_Symbol, TimeFrame) % StudyPeriod) == 0)
      Train();
  }
```

Далее мы обращаем свой взор на метод обучения моделей Train. Здесь следует обратить внимание, что дообучение осуществляется с учетом опыта, полученного в процессе текущего взаимодействия со средой. В методе обработки тиков мы собирали всю полученную от среды информацию в отдельную траекторию. Однако, данная траектория не добавляется в буфер воспроизведения опыта. Ранее подобную операцию мы осуществляли только по завершении эпизода. Но такой подход не допустим в нашем случае периодического обновления параметров. Ведь он приближает нас к офлайн обучению, когда политика Агента обучается только на фиксированных траекториях предыдущего опыта. И, следовательно, перед началом процесса обучения мы добавим собранные данные в буфер воспроизведения опыта.

С целью предотвращения записи слишком коротких и не информативных траекторий мы ограничим минимальный размер сохраняемой траектории. В приведенном примере я ограничил минимальный размер траектории периода обновления параметров модели.

Если размер накопленной траектории удовлетворяет минимальным требованиям, то мы добавляем её в буфер воспроизведения опыта и пересчитываем накопительную сумму вознаграждений.

Здесь следует обратить внимание, что накопительную сумму вознаграждений мы пересчитываем только для копии траектории, перенесенной в буфер воспроизведения опыта. В исходном буфере накопления информации о текущей траектории вознаграждение должно остаться не пересчитанным. Ведь при последующем взаимодействии с окружающей средой траектория будет дополнена. А, следовательно, при дальнейшем добавлении обновленной траектории повторный пересчет накопительного вознаграждения приведет к задвоению данных. И чтобы не допустить этого, в буфере накопления траектории мы всегда держим не пересчитанное вознаграждение.

```
//+------------------------------------------------------------------+
//| Train function                                                   |
//+------------------------------------------------------------------+
void Train(void)
  {
   int total_tr = ArraySize(Buffer);
   if(Base.Total >= StudyPeriod)
      if(ArrayResize(Buffer, total_tr + 1) == (total_tr + 1))
        {
         Buffer[total_tr] = Base;
```

```
          Buffer[total_tr].CumRevards();
          total_tr++;
         }
```

Далее следует вспомнить, что размер буфера накопления траектории у нас ограничен константой Buffer_Size. И с целью предотвращения ошибки выхода за пределы массива мы проверяем наличие достаточного количества свободных ячеек в буфере накопления траектории для записи шагов до следующего сохранения траектории. В случае необходимости удаляем несколько наиболее старых шагов.

Обратите внимание, что удаляем данные в первичном буфере накопления траектории. При этом в копии траектории, которую мы сохранили в буфер воспроизведения опыта данная информация сохраняется.

При указании констант и параметров модели следует учесть, чтобы размер буфера траектории позволял сохранить историю как минимум одного периода между дообучениями моделей.

```
   int clear = Base.Total + StudyPeriod - Buffer_Size;
   if(clear > 0)
      Base.ClearFirstN(clear);
```

Затем я добавил еще один дополнительный контроль, который может показаться излишним. Я проверяю буфер воспроизведения опыта на наличие коротких траекторий и при нахождении удаляю их. На первый взгляд, наличие таких траекторий мало вероятно ввиду наличия аналогичного контроля перед добавлением траектории в буфер воспроизведения опыта. Но я все же допускаю возможность каких-либо сбоев при чтении и записи траекторий в файл. И чтобы исключить последующие ошибки мы выполняем данную проверку.

```
//---
   int count = 0;
   for(int i = 0; i < (total_tr + count); i++)
     {
      if(Buffer[i + count].Total < StudyPeriod)
        {
         count++;
         i--;
         continue;
        }
      if(count > 0)
         Buffer[i] = Buffer[i + count];
     }
   if(count > 0)
     {
      ArrayResize(Buffer, total_tr - count);
      total_tr = ArraySize(Buffer);
     }
```

Далее мы организовываем систему циклов обучения моделей. Этот процесс во многом повторяет аналогичный из предыдущей статьи. Внешний цикл организован по числу итераций обучения моделей, указанному во внешних параметрах советника.

В теле цикла мы случайным образом выбираем траекторию и элемент данной траектории, с которого начнем очередную итерацию обучения моделей.

```
   uint ticks = GetTickCount();
//---
   bool StopFlag = false;
   for(int iter = 0; (iter < StudyIters && !IsStopped() && !StopFlag); iter ++)
```

```
      {
       int tr = (int)((MathRand() / 32767.0) * (total_tr - 1));
       int i = (int)((MathRand() * MathRand() / MathPow(32767, 2)) * MathMax(Buffer[tr].
                                                                              MathMin(Buf
       if(i < 0)
         {
          iter--;
          continue;
         }
```

Затем мы очистим буфера эмбединга моделей и вектор предыдущих действий Актера.

```
      vector<float> Actions = vector<float>::Zeros(NActions);
      AgentStudy.Clear();
      RTGStudy.Clear();
```

На этом этапе мы завершили подготовительную работу и можем начать операции непосредственного обучения моделей. Организовываем вложенный цикл обучения.

В теле цикла мы повторяем процесс подготовки буфера исходных данных, аналогичный описанному выше в методе обработки тиков. Последовательность записи данных в буфер полностью повторяется. Только если раньше данные мы запрашивали у терминала, то сейчас мы берем их из буфера воспроизведения опыта.

```
      for(int state = i; state < MathMin(Buffer[tr].Total - 2, int(i + HistoryBars *
        {
         //--- History data
         bState.AssignArray(Buffer[tr].States[state].state);
         //--- Account description
         float prevBalance = (state == 0 ? Buffer[tr].States[state].account[0] : Buffer
         float prevEquity = (state == 0 ? Buffer[tr].States[state].account[1] : Buffer[
         bState.Add((Buffer[tr].States[state].account[0] - prevBalance) / prevBalance);
         bState.Add(Buffer[tr].States[state].account[1] / prevBalance);
         bState.Add((Buffer[tr].States[state].account[1] - prevEquity) / prevEquity);
         bState.Add(Buffer[tr].States[state].account[2]);
         bState.Add(Buffer[tr].States[state].account[3]);
         bState.Add(Buffer[tr].States[state].account[4] / prevBalance);
         bState.Add(Buffer[tr].States[state].account[5] / prevBalance);
         bState.Add(Buffer[tr].States[state].account[6] / prevBalance);
         //--- Time label
         double x = (double)Buffer[tr].States[state].account[7] / (double)(D'2024.01.01
         bState.Add((float)MathSin(2.0 * M_PI * x));
         x = (double)Buffer[tr].States[state].account[7] / (double)PeriodSeconds(PERIOD
         bState.Add((float)MathCos(2.0 * M_PI * x));
         x = (double)Buffer[tr].States[state].account[7] / (double)PeriodSeconds(PERIOD
         bState.Add((float)MathSin(2.0 * M_PI * x));
         x = (double)Buffer[tr].States[state].account[7] / (double)PeriodSeconds(PERIOD
         bState.Add((float)MathSin(2.0 * M_PI * x));
         //--- Prev action
         bState.AddArray(Actions);
```

После сбора первой части исходных данных мы осуществляем прямой проход модели генерации RTG. И тут же осуществляем прямой проход с целью минимизации ошибки к фактически полученному вознаграждению. Таким образом мы выстраиваем авторегрессионную модель прогнозирования возможного вознаграждения по предшествующей траектории состояния и действий.

```
            //--- Return to go
            if(!RTGStudy.feedForward(GetPointer(bState)))
              {
               PrintFormat("%s -> %d", __FUNCTION__, __LINE__);
               StopFlag = true;
               break;
              }
            Result.AssignArray(Buffer[tr].States[state + 1].rewards);
            if(!RTGStudy.backProp(Result))
              {
               PrintFormat("%s -> %d", __FUNCTION__, __LINE__);
               StopFlag = true;
               break;
              }
```

Для целей обучения политики Агента в буфер исходных данных вместо прогнозного RTG мы указываем фактически полученное вознаграждение и осуществляем прямой проход.

```
            //--- Policy Feed Forward
            bState.AddArray(Buffer[tr].States[state + 1].rewards);
            if(!AgentStudy.feedForward(GetPointer(bState), 1, false, (CBufferFloat*)NULL))
              {
               PrintFormat("%s -> %d", __FUNCTION__, __LINE__);
               StopFlag = true;
               break;
              }
```

Обучение политики Агента осуществляется на минимизацию ошибки между прогнозным и фактически осуществленным действием, которое привело к получению вознаграждения.

```
            //--- Policy study
            Actions.Assign(Buffer[tr].States[state].action);
            vector<float> result;
            AgentStudy.getResults(result);
            Result.AssignArray(CAGrad(Actions - result) + result);
            if(!AgentStudy.backProp(Result, (CBufferFloat*)NULL))
              {
               PrintFormat("%s -> %d", __FUNCTION__, __LINE__);
               StopFlag = true;
               break;
              }
```

Это позволяет выстроить авторегрессионную модель выбора оптимального действия для получения желаемого вознаграждения в контексте посещенных ранее состояний и совершенных действий Агента.

После успешного выполнения итераций обучения моделей мы информируем пользователя о ходе выполнения операций обучения и переходим к следующей итерации цикла обучения моделей.

По завершению всех итерации системы вложенных циклов процесса обучения моделей мы очищаем поле комментариев на графике и переносим параметры из обучаемых моделей в модели взаимодействия с окружающей средой. В приведенном примере я полностью копирую данные весовых коэффициентов. Тем самым эмитирую использование одной модели для обучения и эксплуатации. Однако допускаю и эксперименты с различными коэффициентами копирования данных.

```
    Comment("");
//---
```

```
    Agent.WeightsUpdate(GetPointer(AgentStudy), 1.0f);
    RTG.WeightsUpdate(GetPointer(RTGStudy), 1.0f);
//---
  }
```

На этом мы завершаем рассмотрение алгоритма советника онлайн обучения Трансформера решений, а с полным кодом советника и всех его методов можно ознакомиться во вложении.

Обратите внимание, что советник «...\DoC\OnlineStudy.mq5» расположен в подкаталоге «DoC» с советниками из предыдущей статьи. Я не стал его выделять в отдельный подкаталог, так как функционально он осуществляет дообучение моделей, обученных офлайн советниками из предыдущей статьи. Тем самым мы сохраняем целостность набора файлов обучения моделей.

Так же во вложении Вы можете найти все программы, используемые как в текущей, так и в предыдущих статьях.

3. Тестирование

Выше мы рассмотрели теоретические аспекты метода Online Decision Transformer и построили свою, немного вольную, интерпретацию предложенного метода. Следующим этапом идет тестирование проделанной работы. Фактически мы осуществляем тонкую настройку моделей из предыдущей статьи. Для этих целей мы в тестере стратегий осуществляем цикл одиночных прогонов нашего нового советника по истории обучающих данных.

Напомню, что в предыдущей статье мы осуществляли офлайн обучение моделей на исторических данных за первые 7 месяцев 2023 года. На этом же историческом промежутке мы и осуществляем тонкую настройку моделей.

В процессе тонкой настройки моделей ODT позволил повысить общую доходность моделей. На тестовой выборке за август 2023 года модель смогла заработать около 10% прибыли. График доходности не идеален, но на нем уже прослеживаются некоторые тенденции.

Результаты проведенного тестирования обученной модели представлены выше. В целом за период тестирования была совершена 271 сделка. И 128 из них было закрыто с прибылью, что составило более 47%. Как видно, доля прибыльных сделок немного меньше убыточных. Но максимальная прибыльная сделка на 26% превышает максимальный убыток. А средняя прибыльная сделка более чем на 20% превышает среднюю убыточную сделку. Все это позволило повысить профит-фактор работы модели до уровня 1.10.

Заключение

В данной статье мы продолжили рассмотрение вариантов повышения эффективности метода Трансформер решений (Decision Transformer) и познакомились с алгоритмом тонкой настройки моделей в режиме онлайн обучения Online Decision Transformer (ODT). Данный метод позволяет повышать эффективность работы моделей обученных в офлайн режиме и дает возможность Агентам

адаптироваться к изменяющейся среде, тем самым улучшать свои политики через взаимодействие с окружающей средой.

В практической части статьи мы реализовали метод средствами MQL5 и осуществили онлайн обучение моделей из предыдущей статьи. Здесь стоит отметить, что оптимизация моделей получена только за счет применения рассмотренного метода ODT. В процессе онлайн обучения мы использовали модели, которые были обучены офлайн в рамках предыдущей статьи. Мы не вносили каких-либо конструктивных изменений в архитектуру моделей. Осуществлялось лишь онлайн дообучение. И это позволило повысить эффективность моделей, что само по себе подтверждает результативность использования метода Online Decision Transformer.

Ещё раз хочу напомнить, что все программы, представленные в статье, предназначены лишь для демонстрации технологии и не готовы для использования в реальной торговле.

Ссылки

- Decision Transformer: Reinforcement Learning via Sequence Modeling
- Online Decision Transformer
- Dichotomy of Control: Separating What You Can Control from What You Cannot
- Нейросети — это просто (Часть 34): Полностью параметризированная квантильная функция
- Нейросети — это просто (Часть 58): Трансформер решений (Decision Transformer—DT)
- Нейросети — это просто (Часть 59): Дихотомия контроля (Dichotomy of Control — DoC)

Программы, используемые в статье

#	Имя	Тип	Описание
1	Research.mq5	Советник	Советник сбора примеров
2	Study.mq5	Советник	Советник обучения агента
3	OnlineStudy.mq5	Советник	Советник онлайн дообучения агента
4	Test.mq5	Советник	Советник для тестирования модели
5	Trajectory.mqh	Библиотека класса	Структура описания состояния системы
6	NeuroNet.mqh	Библиотека класса	Библиотека классов для создания нейронной сети
7	NeuroNet.cl	Библиотека	Библиотека кода программы OpenCL

Следующая статья >>

Dmitriy Gizlyk

Украина 108142

Профессиональное написание программ любой сложности для MT4, MT5, C#.

Другие статьи автора

- Нейросети в трейдинге: Диффузионная генерация торговых планов (Планировщик)
- Нейросети в трейдинге: Диффузионная генерация торговых планов (Сценарный расчёт)
- Нейросети в трейдинге: Диффузионная генерация торговых планов (Вычислительная основа)
- Нейросети в трейдинге: Диффузионная генерация торговых планов (DPCC)
- Нейросети в трейдинге: Управление риском через распределение результатов (Окончание)
- Нейросети в трейдинге: Управление риском через распределение результатов (Объект верхнего уровня)
- Нейросети в трейдинге: Управление риском через распределение результатов (Вычислительная основа)

Подпишитесь и узнавайте о
популярных статьях и кодах